CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment 7

Student Name: Geetika
UID: 23BCS12885
Branch: BE CSE
Section/Group: KRG 3A
Semester: 6th
Date of Performance: 09/03/26
Subject Name: Full Stack Development
Subject Code: 23CSH-309

Aim:
To understand and implement JPA entity mapping, advanced querying, fetch strategies, and database performance optimization techniques in a backend application by building efficient data access layers using Spring Boot, JPA, and Hibernate.

Objective:
1. Implement JPA Entity Mapping
- Create entity classes using JPA annotations.
- Establish One-to-Many relationships between entities.
- Understand how ORM maps objects to relational database tables.

2. Perform Advanced Queries
- Write and execute JPQL queries.
- Implement cursor-based pagination for efficient data retrieval.
- Understand the importance of limiting result sets in large datasets.

3. Explore Fetch Strategies
- Implement Lazy and Eager fetching.
- Analyze how different fetch strategies impact application performance.
- Identify and resolve the N+1 query problem.

4. Improve Backend Performance
- Apply database indexing for frequently queried fields.
- Optimize query execution using query tuning techniques.
- Measure and compare performance improvements.

5. Apply Concepts in Project (HealthHub)
- Integrate optimized entity relationships and queries into the HealthHub backend project.
- Evaluate performance impact in real application scenarios.

Implementation/Code:

PatientController.java:

```
package com.example.healthhub.controller;

import com.example.healthhub.dto.*;
import com.example.healthhub.service.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import org.springframework.data.domain.Page;

@RestController
@RequestMapping("/patients")
public class PatientController {

    @Autowired
    private PatientService patientService;

    @PostMapping
    public PatientDTO createPatient(@RequestBody PatientDTO dto)
        { return patientService.savePatient(dto);
    }

    @GetMapping
    public Page<PatientDTO>
            getPatients( @RequestParam int
            page, @RequestParam int size) {
        return patientService.getPatients(page, size);
    }

    @GetMapping("/{id}")
    public PatientDTO getPatient(@PathVariable Long id)
        { return patientService.getPatient(id);
    }
}
```

PatientRepository.java:

```
package com.example.healthhub.repository;

import com.example.healthhub.entity.Patient;
import org.springframework.data.jpa.repository.*;
import org.springframework.data.repository.query.Param;
import org.springframework.data.domain.*;
```

```
import java.util.*;

public interface PatientRepository extends JpaRepository<Patient, Long> {

    @Query("SELECT p FROM Patient p WHERE p.age > :age")
    List<Patient> findPatientsOlderThan(@Param("age") int age);

    @Query("SELECT p FROM Patient p LEFT JOIN FETCH p.appointments")
    List<Patient> findAllWithAppointments();

    @Query("SELECT p FROM Patient p LEFT JOIN FETCH p.appointments WHERE p.id
= :id")
    Optional<Patient> findByIdWithAppointments(@Param("id") Long id);

    Page<Patient> findAll(Pageable pageable);

    Optional<Patient> findByEmail(String email);
}
```

PatientService.java:

```
package com.example.healthhub.service;

import com.example.healthhub.entity.*;
import com.example.healthhub.dto.*;
import com.example.healthhub.repository.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.data.domain.*;

import java.util.*;

@Service
public class PatientService {

    @Autowired
    private PatientRepository patientRepository;

    public PatientDTO savePatient(PatientDTO dto)
        { Patient patient = new Patient();
        patient.setName(dto.getName());
        patient.setEmail(dto.getEmail());
        patient.setAge(dto.getAge());
```

```
        Patient saved = patientRepository.save(patient);
        return convertToDTO(saved);
    }

    public Page<PatientDTO> getPatients(int page, int size)
        { Pageable pageable = PageRequest.of(page, size);
        return patientRepository.findAll(pageable)
                .map(this::convertToDTO);
    }

    public PatientDTO getPatient(Long id) {
        Patient patient = patientRepository.findByIdWithAppointments(id)
                .orElseThrow(() -> new RuntimeException("Patient not found"));
        return convertToDTO(patient);
    }

    public PatientDTO convertToDTO(Patient patient)
        { PatientDTO dto = new PatientDTO();

        dto.setId(patient.getId());
        dto.setName(patient.getName());
        dto.setEmail(patient.getEmail());
        dto.setAge(patient.getAge());

        if (patient.getAppointments() != null)
            { List<String> dates = patient.getAppointments()
                    .stream()
                    .map(Appointment::getDate)
                    .toList();
            dto.setAppointmentDates(dates);
        }

        return dto;
    }
}
```

Output:

POST http://localhost:8080/patients Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
  "name": "Sana",
  "email": "sana@gmail.com",
  "age": 35
}
```

Body Cookies Headers (5) Test Results 200 OK 225 ms 244 B

{} JSON Preview Visualize

```
{
    "age": 35,
    "appointmentDates": null,
    "email": "sana@gmail.com",
    "id": 4,
    "name": "Sana"
}
```

GET http://localhost:8080/patients/3 Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers (5) Test Results 200 OK 660 ms 244 B

{} JSON Preview Visualize

```
{
    "age": 30,
    "appointmentDates": [],
    "email": "david@gmail.com",
    "id": 3,
    "name": "David"
}
```

GET http://localhost:8080/patients?page=0&size=2 Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Body Cookies Headers (5) Test Results 200 OK 47 ms 642 B

JSON Preview Visualize


```
{
    "content": [
        {
            "age": 21,
            "appointmentDates": [],
            "email": "tanureet@gmail.com",
            "id": 1,
            "name": "Tanureet"
        },
        {
            "age": 21,
            "appointmentDates": [],
            "email": "ana@gmail.com",
            "id": 2,
            "name": "Ana"
        }
    ],
    "empty": false,
    "first": true,
    "last": false,
    "number": 0,
    "numberOfElements": 2,
    "pageable": {
        "offset": 0,
        "pageNumber": 0,
        "pageSize": 2,
```


GET http://localhost:8080/patients?page=1&size=2 Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Body Cookies Headers (5) Test Results 200 OK 19 ms 638 B

JSON Preview Visualize


```
{
    "content": [
        {
            "age": 30,
            "appointmentDates": [],
            "email": "david@gmail.com",
            "id": 3,
            "name": "David"
        },
        {
            "age": 35,
            "appointmentDates": [],
            "email": "sana@gmail.com",
            "id": 4,
            "name": "Sana"
        }
    ],
    "empty": false,
    "first": false,
    "last": true,
    "number": 1,
    "numberOfElements": 2,
    "pageable": {
        "offset": 2,
        "pageNumber": 1,
        "pageSize": 2,
```

Learning Outcome:

1. Ability to design and implement JPA entity classes using appropriate annotations and relationships, particularly One-to-Many mappings.

2. Understanding of Object-Relational Mapping (ORM) concepts and how Java objects are mapped to relational database tables.

3. Ability to write and execute advanced JPQL queries and implement efficient data retrieval techniques such as cursor-based pagination.

4. Understanding the importance of optimizing queries by limiting result sets when working with large datasets.

5. Ability to differentiate between Lazy and Eager fetching strategies and analyze their impact on application performance.